



FinPilot AI

FINANCE • CONTROL • DECIDE • GROW

Autonomous Financial Digital Twin & Multi-Agent Decision Operating System

Complete Interview Master Guide & Comprehensive System Reference

Track: Razorpay AI Agent

Architecture: Hybrid Multi-Agent

Version: Production 2.4

01 THE 30-SECOND ELEVATOR PITCH (WHAT TO SAY FIRST)

*"Most financial AI tools describe what already happened by reading past dashboards. **FinPilot AI simulates what happens next** — before an executive has to commit capital. Built on **Razorpay test-mode rails**, FinPilot AI combines a **90-Day Financial Digital Twin**, an **Autonomous Multi-Agent Decision Copilot**, a **Merchant Growth Engine with Smart Upsells**, and an **AI-to-AI Shopping Protocol** with zero-leakage role-based security."*

Key Innovation #1: Digital Twin Simulation

Predicts day-by-day cash runway, burn velocity, and deficit risks 90 days forward under what-if scenarios.

Key Innovation #2: AI-to-AI Commerce

Machine-readable store manifest (`/v1/commerce/ai-manifest`) enabling external buyer bots to negotiate within spending caps.

The Traditional Finance Problem	How FinPilot AI Fixes It
1. Retrospective Dashboards: Only show historical data after overspending occurs.	Proactive Digital Twin: Simulates counterfactuals 90 days out before money is disbursed.
2. LLM Hallucination Risk: Blindly trusting AI tokens with arithmetic causes financial errors.	Disciplined AI Judgment: Exact Python math for GST & balances; AI reserved for semantic reasoning.
3. Friction in Merchant Sales: Slow manual checkouts and lack of machine-to-machine commerce.	Autonomous AI-to-AI Shopping & Conversational Cart with 1-click Razorpay payment links.
4. Notification Noise & Data Leakage: Global feeds expose sensitive budgets across roles.	Strict Recipient Isolation: Notifications and requests sent strictly to intended user IDs.

Pillar 1: 🧬 Financial Digital Twin & What-If Studio

Maintains an in-memory clone of enterprise cash reserves (₹8.43M), department burn rates, and vendor liabilities.

- **90-Day Step-by-Step Differential Simulation:** Computes daily cash continuity ($\$Cash_{\{i\}} = Cash_{\{i-1\}} + Inflow_i - Outflow_i$).
- **Interactive Multiplier Sliders:** Allows executives to simulate burn spikes, 14-day vendor payment deferrals, or Q3 headcount additions.
- **4-Factor Decision Scorer (0–100):** Evaluates Budget Impact, Policy Compliance, Variance Stability, and Approval Risk.

Pillar 2: 🤖 7-Agent Autonomous Multi-Agent Pipeline

Coordinates 7 specialized sub-agents executing deterministic hand-offs:

1. Intent & RBAC Agent: Validates role boundaries.

2. Retrieval Agent: Extracts facts with lineage tracking.

3. Analysis Agent: Calculates daily burn velocity & GST.

4. Risk Agent: Computes multi-factor risk scores.

5. Simulation Agent: Runs forward Digital Twin projections.

6. Causal Agent: Pinpoints root cause correlations.

7. Narrator Agent: Synthesizes structured 5-part role-tailored report: (1) What Happened, (2) Why It Matters, (3) What Should Be Done, (4) Who Needs to Act, (5) What Happens Next.

Pillar 3: 🛒 Merchant Growth & AI-to-AI Shopping Protocol

- **Universal Agent Manifest:** Machine-readable JSON-LD / Schema.org protocol at ``/v1/commerce/ai-manifest``.
- **Bounded AI-to-AI Purchasing:** External AI buyer bots negotiate tiered discounts (up to 15%) within hard spending token limits.
- **Conversational In-App Checkout:** Natural language cart assistant with real-time 18% GST and 1-click Razorpay test payment links.
- **Autonomous Campaign Orchestrator:** Deploys growth campaigns synchronized with marketing budget and forecasts ROAS (4.0x).

Pillar 4: 🏢 **Role-Based User-Targeted Notification Center**

- **Strict Recipient Isolation:** Filtered at database layer (``recipientUserId == current_user.id``). Zero cross-user notification leaks.
- **2-Way Threaded Conversations:** Live chat bubble modal with reply inputs between CFO, Managers, and Department Heads.
- **New Request Compose Modal:** Dynamic eligible recipient directory routing.
- **Direct Entity Deep-Linking:** 1-click navigation to referenced invoices, budgets, or tax reconciliation screens.

Pillar 5: 🛡️ **Four-Tier Failure Resilience Engine**

1. Gateway Timeout: Exponential backoff with idempotency key and offline queue.

2. Malformed Payload: Schema validation with field-level corrective hints.

3. Budget Cap Hit: Pre-disbursement freeze with automated escalation ticket.

4. Token Limit Breach: Transaction paused for human override authorization.

04 DISCIPLINED AI JUDGMENT: KNOWING WHEN NOT TO USE AI

Interviewers love to test candidates on whether they understand the limits of LLMs. Here is the exact architectural boundary:

Where We DO NOT Use AI:

- **Ledger & Tax Arithmetic:** GST (18%), discounts, and burn rates use pure Python floating-point math to eliminate hallucinations.
- **Security & RBAC Ceilings:** ₹50K HITL threshold and token limits are validated deterministically in Pydantic models.
- **Audit Chaining:** SHA-256 cryptographic hashing links compliance records immutably.

Where We DO Use AI:

- **Natural Language Understanding:** Extracting semantic purchase intent from conversational requests.
- **Causal Root Cause Correlation:** Linking operational signals with spending anomalies.
- **Multi-Agent Role-Aware Narration:** Tailoring executive explanations for CFOs vs Auditors.

05 5-MINUTE LIVE INTERVIEW PITCH SCRIPT (WHAT TO CLICK & SAY)

Minute 1: Login & Executive Overview (Tab 1: Control Center)

"I'm logging in as the CFO Vikramaditya. Notice the live ₹8.43M liquidity, 82/100 decision score, and dynamic department burn telemetry. Every action here is governed by our Financial Digital Twin."

Minute 2: 90-Day Digital Twin Simulation (Tab 2: What-If Studio)

"Let's test what happens if Engineering spend increases by 15% and we delay vendor payments by 14 days. Watch the trajectory chart instantly project cash reserves and runway without touching the production ledger."

Minute 3: Multi-Agent Financial Copilot (Tab 8: Copilot Chat)

"I'll click the prompt 'Why is Engineering overspending?'. Our 7-sub-agent orchestrator classifies intent, pulls telemetry, runs causal correlation, and generates a 5-step role-tailored action plan."

Minute 4: Merchant Sales Growth & AI-to-AI Shopping (Tab 13: Commerce Studio)

"Here is our AI commerce engine. I can chat with the cart assistant, get a 15% smart upsell bundle, or launch an autonomous AI buyer bot that negotiates within spending token limits and produces a Razorpay payment link."

Minute 5: Role-Based Notifications & Audit Trail (Bell Icon)

"Finally, clicking the notification bell opens our user-targeted notification center with 2-way threads, 1-click entity routing, and SHA-256 cryptographically chained audit logging."

Q1: How do you prevent AI hallucinations in financial calculations?

Answer: We enforce strict architectural boundaries. All arithmetic (18% GST, burn rate calculations, subtotal sums, and budget balances) is computed deterministically in Python. The LLM is only used for intent parsing, causal correlation, and semantic narrative formulation.

Q2: How does the AI-to-AI Shopping protocol work?

Answer: We expose a machine-readable JSON-LD catalog manifest at `/v1/commerce/ai-manifest`. External buyer agents send purchase requests with an authorized `X-Spending-Token-Limit` header. Our merchant agent validates the ceiling, applies bulk discount rules within policy constraints (max 15%), and returns a test-mode Razorpay payment link.

Q3: How does the Financial Digital Twin simulate 90 days forward?

Answer: It takes an in-memory snapshot of liquid reserves, department budgets, and scheduled invoices, then applies day-by-day differential equations with modifier multipliers (burn spikes, payment delays, headcount changes) to project daily cash continuity and runway days.

Q4: How do you enforce strict role-based notification isolation?

Answer: Notifications are filtered at the service query layer against `recipientUserId == current_user.id` or explicit `observerIds`. Senders, recipients, and observers are isolated so no department head or auditor can see private CFO requests.

Q5: What happens when Razorpay API drops or times out?

Answer: Our 4-Tier Failure Resilience Engine kicks in with an exponential backoff retry mechanism (3 attempts with jitter) using idempotency keys. If the gateway remains unresponsive, the transaction is queued in an offline state and an escalation ticket is dispatched.

Q6: What is the tech stack and how is it tested?

Answer: Backend: Python 3.11, FastAPI, Pydantic, LangChain/LangGraph, SQLite/In-memory store. Frontend: Tailwind CSS, Vanilla JS, Lucide icons, Chart.js. We have 7 automated test suites with 100% pass rate including Chrome CDP browser automation tests.